

W05-asm2---Performance Report

Parallel Bucket Sort (Scatterv / Alltoallv / Gatherv)

Test environment: CTCP cluster, 32 logical cores available. **Benchmark design:** work-per-process held fixed at 5,000,000 numbers; N scales proportionally with numproc (5M, 10M, 20M, 40M, 80M for p = 1, 2, 4, 8, 16) — this is the correct setup for testing **Gustafson's Law**, as opposed to Amdahl's Law (which instead holds N fixed).

1. Correctness

Every configuration tested (p = 1, 2, 4, 8, 16, three repeats each) reported `sorted_ok = YES`. The parallel bucket sort implementation produces a correctly sorted array in all cases, confirming that the Scatterv → classify → Alltoallv → sort → Gatherv pipeline preserves data correctly regardless of process count or problem size.

2. Raw timing data (average of 3 runs per configuration, seconds)

numproc	N	scatter	classify	alltoallv	sort	gatherv	total
1	5,000,000	0.1280	0.0782	0.0506	0.7008	0.0264	0.9839
2	10,000,000	0.2518	0.0800	0.0566	0.7797	0.0554	1.2235
4	20,000,000	0.4693	0.0766	0.0611	0.8758	0.1137	1.5965
8	40,000,000	1.0212	0.0781	0.0673	1.1123	0.2381	2.5171
16	80,000,000	2.0906	0.0765	0.1040	1.1420	0.4822	3.8952

3. Gustafson's Law Analysis

3.1 Scaled speedup

Since work-per-process is held constant, an ideally scalable program should show **roughly constant** `total_time` as p increases, giving a scaled speedup close to linear:

$$S_{scaled}(p) = \frac{p \cdot T(1)}{T(p)}$$

numproc	avg total_time (s)	S_scaled(p)	ideal (=p)	implied overhead α
1	0.9839	1.000	1	0.0%
2	1.2235	1.608	2	39.2%
4	1.5965	2.465	4	51.2%
8	2.5171	3.127	8	69.6%
16	3.8952	4.042	16	79.7%

where α is the implied overhead/serial fraction from the Gustafson-Barsis relation $S_{scaled}(p) = p - \alpha(p-1)$.

3.2 Do we get the results expected from Gustafson's Law?

No — not overall. Under ideal Gustafson scaling, `total_time` should stay close to the $p=1$ baseline (~0.98s) at every process count, since each process always has the same amount of local work (5,000,000 numbers). Instead, `total_time` grows nearly **4×** from $p=1$ to $p=16$ (0.98s → 3.90s), and the scaled speedup at $p=16$ is only 4.04 instead of the ideal 16 — the implied overhead fraction α climbs from 0% to almost 80%.

3.3 What is causing this?

Breaking the total down by phase reveals the cause clearly:

Phase	p=1 → p=16 growth	Behaviour
classify_time	0.0782 → 0.0765 (flat)	Obeys Gustafson's Law
alltoallv_time	0.0506 → 0.1040 (~2×)	Mild growth
sort_time	0.7008 → 1.1420 (~1.6×)	Moderate growth
scatter_time	0.1280 → 2.0906 (~16×)	Dominant bottleneck
gatherv_time	0.0264 → 0.4822 (~18×)	Second bottleneck

scatter_time and **gatherv_time** are the primary reason Gustafson's Law is violated. Both operations are inherently centralised at the master:

- **MPI_Scatterv** requires the **master alone** to hold and transmit the *entire* array of N numbers, no matter how many processes are receiving it. As N grows proportionally with p (by the Gustafson benchmark design), the master's total data-transmission workload grows by the same factor — there is no way to parallelise "the master's own job of serialising and sending out N numbers." This is a genuine, unavoidable serial bottleneck created by the requirement that "only the master process initializes the array."
- **MPI_Gatherv** has the mirror-image problem: the master alone must receive and reassemble all N sorted numbers at the end, regardless of p.

Both costs scale with **total data volume $O(N)$** , and since $N = p \times$ (fixed work-per-process), this is equivalent to **$O(p)$** growth — which is exactly what the ~16–18× growth over a 16× increase in p reflects. This is a **structural property of the master-distributes/master-collects design**, not a flaw in the bucket sort algorithm itself.

3.4 Are there any parts of the program that do obey Gustafson's Law?

Yes — classify_time is essentially constant across every configuration (0.076–0.080s throughout). This is exactly what Gustafson's Law predicts for genuinely local, embarrassingly-parallel work: each process classifies only its own N/p elements into small buckets, with **no inter-process communication** at all in this phase, so its cost depends only on the (fixed) amount of local work — never on p.

alltoallv_time is the next-best behaved phase, growing only modestly (~2× over a 16× increase in p) compared to scatter/gather's ~16–18×. This is because Alltoallv distributes its communication cost **across all processes simultaneously** rather than funnelling

everything through a single master — each process only sends/receives roughly its own share of data. The mild residual growth is most likely due to the increasing **number of point-to-point exchanges** implicit in an all-to-all pattern (on the order of $p(p-1)$ messages), which grows even though the data volume per process stays fixed.

`sort_time`'s moderate growth ($\sim 1.6\times$) is a smaller effect worth noting: although the *expected* large-bucket size is constant ($\approx 5,000,000/1$ for $p=1$ vs $\approx 80,000,000/16$ for $p=16$ — the same in theory), the reported time is measured only on the **master's own bucket**. With random data, bucket sizes fluctuate around the expected value, and with more buckets (larger p), statistical variance means the master's actual bucket size can deviate further from the mean — a mild load-imbalance effect distinct from the scatter/gatherv bottleneck.

4. Methodological note (for future improvement)

All timings reported are measured **locally by the master process only** (rank 0's own `t1-t0`, `t2-t1`, etc.). This is not necessarily the bottleneck process — under random data distribution, a different rank could have a larger bucket and take longer to classify/sort/communicate. A more rigorous measurement would use `MPI_Reduce` with `MPI_MAX` on each phase's local timing across all ranks, giving the true worst-case (bottleneck) time per phase rather than just the master's own experience. This does not change the qualitative conclusions above (scatter/gatherv dominate), but would make the absolute numbers more representative of overall wall-clock behaviour, especially for `classify_time`, `sort_time` and `alltoallv_time`, which are measured per-rank and could vary between processes.

5. Summary

- **Correctness:** 100% of test runs (15/15) produced a correctly sorted array.
- **Gustafson's Law:** Not observed overall — `total_time` grows $\sim 4\times$ from $p=1$ to $p=16$ instead of staying flat, giving a scaled speedup of only 4.04 at $p=16$ (vs. the ideal 16).
- **Root cause:** The `Scatterv` / `Gatherv` phases are centralised at the master and scale with total data volume $O(N) = O(p \times \text{constant})$, making them the dominant bottleneck ($\sim 16\text{--}18\times$ growth) as the problem is scaled up with p .

- **Parts that do obey Gustafson's Law:** `classify_time` (purely local, no communication) stays essentially constant; `alltoallv_time` grows only mildly since its communication cost is distributed across all processes rather than centralised at the master.